Capstone 1 — Domain A: Pre-Auth Status Checker

Build your first agent: a single-tool conversational assistant that checks healthcare pre-authorization status and responds with clear, actionable next steps.

Project Brief

BUSINESS CONTEXT

Every day, healthcare provider offices spend hours on the phone with insurance companies checking the status of prior authorization requests. A front-desk coordinator dials the payer, navigates an IVR phone tree, waits on hold for 15–45 minutes, reads off a reference number, and manually transcribes the response into the patient's chart. Multiply that by 30–50 auths per day across a busy orthopedic practice, and you have an entire full-time role dedicated to hold music.

The pain is threefold: the process is slow (up to an hour per auth check), error-prone (manual transcription leads to missed deadlines and wrong next-steps), and frustrating for staff who would rather help patients face-to-face. When an auth status changes from "pending" to "info-requested" and nobody notices, the deadline passes, the auth is denied, and the patient's surgery gets delayed by weeks.

Your agent replaces this phone-based workflow: the user provides a pre-auth reference number, the agent calls a payer status API (mock), and returns a clear, plain-English summary with actionable next steps — "Approved: you may schedule the procedure" or "Info Requested: upload the MRI report by March 15." One query, immediate answer, zero hold time.

WHAT YOU WILL BUILD

A conversational agent with one tool (`get_preauth_status`) that:

- Accepts a pre-authorization reference number from the user
- Calls the tool to retrieve mock payer status data
- Summarizes the status in plain English with context-appropriate next steps
- Handles errors gracefully (invalid format, not found, system unavailable)
- Maintains conversation context across multiple turns

Skills practiced: Tool definitions (M05), structured output (M04), conversation management (M08), the agentic tool-use loop, and error handling.

Stretch goal: Add a second tool (`lookup_cpt_code`) to look up CPT code descriptions so the agent can explain what procedure is being authorized.

Prerequisites

BEFORE YOU START

This is a ★☆☆☆☆ **(1 of 5)** capstone — the easiest project in the series. You need working knowledge of three modules:

- M03: Prompt Engineering — system prompts, role-setting, and instruction design. You will write a system prompt that constrains the agent to status-checking only.
- M04: Structured Output — getting Claude to return predictable JSON. The tool returns structured data that the agent interprets for the user.
- M05: Function Calling (Tool Use) — defining tool schemas, handling `tool_use` responses, and sending `tool_result` blocks back. This is the core mechanic of the entire capstone.

If you have not completed M03–M05, go back and finish them first. This capstone directly applies concepts from those modules.

TIME ESTIMATE & DIFFICULTY

30–45 minutes for the core project (Steps 1–5). The stretch goals and extensions are optional and may add another 30–60 minutes if you choose to tackle them.

You will create 3 files totaling about 250 lines of Python. No external services, databases, or Docker required — everything runs locally with mock data.

Domain Glossary

Prior Authorization (Pre-Auth)

A requirement by insurance companies that providers must get approval before delivering certain services. The payer reviews clinical criteria to determine medical necessity.

CPT Code

Current Procedural Terminology — a 5-digit code identifying a medical procedure or service (e.g., 27447 = Total Knee Arthroplasty). Maintained by the AMA.

ICD-10 Code

International Classification of Diseases, 10th Revision — a code identifying a diagnosis (e.g., M17.11 = Primary osteoarthritis, right knee). Used alongside CPT codes in authorization requests.

Payer

The insurance company or health plan that pays for the patient's care. Examples: Aetna, UnitedHealthcare, Blue Cross Blue Shield. Each payer has its own authorization policies.

Provider

The healthcare professional or facility requesting authorization — the doctor, hospital, or clinic that wants to perform the procedure and get paid for it.

Medical Necessity

The clinical justification for a procedure. Payers require evidence that the service is clinically appropriate, not experimental, and the most cost-effective option.

Clinical Reviewer

A licensed healthcare professional (usually an MD or RN) employed by the payer who reviews pre-auth requests and makes approval/denial determinations.

HIPAA

Health Insurance Portability and Accountability Act — federal law governing the privacy and security of Protected Health Information (PHI). Any system handling patient data must comply.

Architecture

The architecture is deliberately simple: a single user-facing agent with one tool. This lets you focus on mastering the tool-use loop before adding complexity in later capstones.

ARCHITECTURE — SINGLE-TOOL AGENT PATTERN

THE TOOL-USE LOOP

PRE-AUTH STATUS FLOW — POSSIBLE OUTCOMES

Environment Setup

Copy and paste these commands to set up your project from scratch. Everything runs locally — no Docker, no cloud services, no database.

SYSTEM REQUIREMENTS

- **Python 3.10+** — check with `python --version`. The `match` / `case` syntax and modern type hints used in this capstone require 3.10 or later.
- **Node.js 18+** (optional) — only needed if you follow the Node.js/TypeScript code tabs.
- An **Anthropic API key** — get one at console.anthropic.com.

Python Setup

MACOS / LINUX · WINDOWS

MACOS / LINUX

```
# Create the project directory and virtual environment
mkdir preauth-agent && cd preauth-agent
python3 -m venv venv && source venv/bin/activate

# Install dependencies
pip install anthropic pytest

# Set your API key (replace with your real key)
export ANTHROPIC_API_KEY=your-key-here
```

WINDOWS

```
# Create the project directory and virtual environment
# (PowerShell 5.1 does not support && chaining — run line-by-line)
mkdir preauth-agent
cd preauth-agent
python -m venv venv
venv\Scripts\Activate.ps1

# Install dependencies
pip install anthropic pytest

# Set your API key (replace with your real key)
# PowerShell:
$env:ANTHROPIC_API_KEY = "your-key-here"
# Or in cmd.exe:
# set ANTHROPIC_API_KEY=your-key-here
```

Verify Installation

VERIFICATION COMMAND

VERIFICATION COMMAND

```
python -c "import anthropic; print('anthropic', anthropic.__version__); c = anthropic.Anthropic(); print('API key configured:', bool(c.api_key))"
```

Expected Output

```
anthropic 0.39.0
API key configured: True
```

✅ **Checkpoint**

If you see the version number and `API key configured: True`, your environment is ready. If you see `ModuleNotFoundError`, run `pip install anthropic` again. If you see `API key configured: False`, re-run the `export` (macOS/Linux), `$env:` (PowerShell), or `set` (cmd.exe) command with your actual key.

Node.js / TypeScript Setup (Alternative)

If you prefer to follow the Node.js code tabs instead of Python, use the setup below.

NODE.JS / TYPESCRIPT

NODE.JS / TYPESCRIPT

```
# Create the project directory and initialize
# (run line-by-line on PowerShell 5.1 which does not support &&)
mkdir preauth-agent
cd preauth-agent
npm init -y

# Install the Anthropic SDK
npm install @anthropic-ai/sdk

# Install TypeScript tooling
npm install --save-dev typescript ts-node @types/node
npx tsc --init

# Set your API key (replace with your real key)
# macOS / Linux:
export ANTHROPIC_API_KEY=your-key-here
# Windows PowerShell:
# $env:ANTHROPIC_API_KEY = "your-key-here"
```

Verify Node.js Installation

VERIFICATION COMMAND

VERIFICATION COMMAND

```
npx ts-node -e "import Anthropic from '@anthropic-ai/sdk'; const c = new Anthropic(); console.log('API key configured:', !!c.apiKey);"
```

Expected Output

```
API key configured: true
```

File Structure

By the end of this capstone, your project directory will contain these files:

```
preauth-agent/
├── mock_tools.py       # Mock payer API + CPT lookup (Step 1)
├── agent.py            # Main agent with tool-use loop (Steps 2-4)
├── test_agent.py       # Unit tests for mock tools (Step 5)
├── requirements.txt    # anthropic>=0.30.0, pytest
└── .env.example        # ANTHROPIC_API_KEY=your-key-here
```

requirements.txt

Create a `requirements.txt` so anyone cloning the project can install dependencies in one command with `pip install -r requirements.txt`:

REQUIREMENTS.TXT

REQUIREMENTS.TXT

```
anthropic>=0.30.0
pytest>=7.0.0
```

You will create each file one at a time in the build guide below. The Node.js/TypeScript versions (`agent.ts` , `mock_tools.ts`) are provided as reference implementations in each step's code tabs.

Mock Data Specification

Your mock payer API returns this JSON structure. Study the fields carefully — your agent needs to extract the relevant information and present it in plain English based on the `status` value.

JSON SCHEMA

JSON SCHEMA

```
{
  "reference_id": "PA-2024-00847",
  "status": "info-requested",         // approved | pending | denied | info-requested
  "patient_name": "Jane Doe",
  "procedure_code": "27447",
  "procedure_description": "Total Knee Arthroplasty",
  "diagnosis_codes": ["M17.11"],       // ICD-10 codes
  "payer": "Aetna",
  "submitted_date": "2024-03-01",
  "last_updated": "2024-03-10",
  "clinical_reviewer_notes": "Requesting operative report and 6-month conservative treatment documentation.",
  "timeline": {
    "response_deadline": "2024-03-20",
    "days_remaining": 10
  },
  "assigned_reviewer": "Dr. Smith, MD"
}
```

Sample Records (All Four Statuses)

The mock tool ships with these pre-loaded records so you can test every code path:

SAMPLE DATA

SAMPLE DATA

```
// PA-2024-00847 — Info Requested (above)
// PA-2024-01122 — Approved
{
  "reference_id": "PA-2024-01122",
  "status": "approved",
  "patient_name": "Robert Chen",
  "procedure_code": "70553",
  "procedure_description": "MRI Brain w/ and w/o Contrast",
  "diagnosis_codes": ["G43.909"],
  "payer": "UnitedHealthcare",
  "submitted_date": "2024-02-20",
  "last_updated": "2024-02-28",
  "clinical_reviewer_notes": "Meets medical necessity criteria. Approved for 1 study.",
  "timeline": { "response_deadline": null, "days_remaining": null },
  "assigned_reviewer": "Dr. Patel, MD"
}

// PA-2024-00519 — Denied
{
  "reference_id": "PA-2024-00519",
  "status": "denied",
  "patient_name": "Maria Garcia",
  "procedure_code": "29881",
  "procedure_description": "Knee Arthroscopy with Meniscectomy",
  "diagnosis_codes": ["M23.211"],
  "payer": "Cigna",
  "submitted_date": "2024-01-15",
  "last_updated": "2024-02-01",
  "clinical_reviewer_notes": "Conservative treatment not attempted for minimum 6 weeks. Does not meet InterQual
criteria.",
  "timeline": { "response_deadline": null, "days_remaining": null },
  "assigned_reviewer": "Dr. Johnson, MD"
}

// PA-2024-02001 — Pending
{
  "reference_id": "PA-2024-02001",
  "status": "pending",
  "patient_name": "David Wilson",
  "procedure_code": "27130",
  "procedure_description": "Total Hip Arthroplasty",
  "diagnosis_codes": ["M16.11"],
  "payer": "Blue Cross Blue Shield",
  "submitted_date": "2024-03-08",
  "last_updated": "2024-03-08",
  "clinical_reviewer_notes": null,
  "timeline": { "response_deadline": "2024-03-22", "days_remaining": 14 },
  "assigned_reviewer": null
}
```

🎯 **What Just Happened?**

You now have four test records covering every status branch. When you build the agent, each status triggers a different response pattern: approved → scheduling instructions, denied → appeal process, info-requested → deadline warning with upload instructions, pending → estimated timeline.

Step-by-Step Build Guide

You have the architecture, data schema, and environment ready. Now let's build the agent file by file. Each step creates one complete file that you can test immediately before moving on.

Step 1: Create the Mock Payer API

What & Why: Before building the agent itself, you need something for the agent to call. This file simulates a payer status API and a CPT code lookup — the same kind of service a real insurance company would expose. By building the mock first, you can test the agent without needing real credentials or network access.

Create a new file called `mock_tools.py` with the following complete code:

PYTHON · NODE.JS

PYTHON

```python
IMPORT re

# ── Mock database ──────────────────────────────────────
PREAUTH_DB = {
    "PA-2024-00847": {
        "reference_id": "PA-2024-00847",
        "status": "info-requested",
        "patient_name": "Jane Doe",
        "procedure_code": "27447",
        "procedure_description": "Total Knee Arthroplasty",
        "diagnosis_codes": ["M17.11"],
        "payer": "Aetna",
        "submitted_date": "2024-03-01",
        "last_updated": "2024-03-10",

    # … (full source in the desktop HTML) …

        RETURN {
            "error": "NOT_FOUND",
            "message": f"CPT code '{cpt_code}' not found in database.",
        }

    RETURN record
```

NODE.JS

```javascript
// mock_tools.ts — Mock payer status API for Capstone 1-A

// ── Mock database ──────────────────────────────────────
CONST PREAUTH_DB: Record<string, any> = {
  "PA-2024-00847": {
    reference_id: "PA-2024-00847",
    status: "info-requested",
    patient_name,
    procedure_code,
    procedure_description,
    diagnosis_codes,
    payer,
    submitted_date: "2024-03-01",
    last_updated: "2024-03-10",

    # … (full source in the desktop HTML) …

  IF (!record) {
    RETURN { error, message: `CPT code '${cptCode}' not found.` };
  }

  RETURN record;
}
```

Run Command (Step 1)

TEST COMMAND

TEST COMMAND

```
python -c "from mock_tools import get_preauth_status, lookup_cpt_code; import json;
print(json.dumps(get_preauth_status('PA-2024-01122'), indent=2))"
```

Expected Output

```
{
  "reference_id": "PA-2024-01122",
```

```
  "status": "approved",
  "patient_name": "Robert Chen",
  "procedure_code": "70553",
  "procedure_description": "MRI Brain w/ and w/o Contrast",
  ...
}
```

✅ **Checkpoint — Step 1**

If you see a JSON object with `"status": "approved"`, the mock tool is working. If you see `ModuleNotFoundError`, make sure you are running from inside the `preauth-agent/` directory. If the import fails, double-check that the file is named exactly `mock_tools.py` (with an underscore, not a hyphen).

TROUBLESHOOTING — STEP 1

`ModuleNotFoundError: No module named 'mock_tools'` — You are not in the `preauth-agent/` directory. Run `cd preauth-agent` first.

`SyntaxError: invalid syntax` — Check that you copied the entire file including the import at the top (`import re`). A missing import causes downstream syntax errors.

Output is `{"error": "NOT_FOUND", ...}` — You changed the test reference ID. Use exactly `PA-2024-01122` to match a pre-loaded record.

Step 2: Create the Agent with Tool Definitions

What & Why: This is the main file — the agent that talks to Claude. It defines two tool schemas (one for status lookup, one for CPT code descriptions), sets a system prompt that constrains the agent to read-only status checks, and implements the tool-use loop that sends messages to Claude, detects tool calls, executes them, and sends results back until Claude produces a final answer.

Create a new file called `agent.py` with the following complete code:

PYTHON · NODE.JS

PYTHON

```python
IMPORT json
IMPORT anthropic
FROM mock_tools import get_preauth_status, lookup_cpt_code

# ── WHAT: Initialize the Anthropic client ──────────────────
# WHY: The client reads ANTHROPIC_API_KEY from the environment.
#      Never hardcode API keys — they rotate and leak easily.
client = anthropic.Anthropic()
MODEL = "claude-sonnet-4-6"

# ── WHAT: Define the tool schema ───────────────────────────
# WHY: Claude uses this JSON schema to decide WHEN to call the
#      tool and WHAT arguments to pass. A clear description is
#      critical — it's the "instruction manual" for the model.

    # … (full source in the desktop HTML) …

            print(f"\n[API Error] {e.message}")
        CATCH:
            print(f"\n[Error] {e}")

IF __name__ == "__main__":
    main()
```

NODE.JS

```javascript
// agent.ts — Pre-Auth Status Checker Agent (Capstone 1-A)
//
// Usage:
//   export ANTHROPIC_API_KEY=your-key-here
//   npx ts-node agent.ts

IMPORT Anthropic from "@anthropic-ai/sdk";
IMPORT * as readline from "readline";
IMPORT { getPreauthStatus, lookupCptCode } from "./mock_tools";

// ── WHAT: Initialize the Anthropic client ──────────────────
// WHY= new Anthropic();
CONST MODEL = "claude-sonnet-4-6";


    # … (full source in the desktop HTML) …

    });
  };
  prompt();
}

main();
```

Run Command (Step 2)

MACOS / LINUX · WINDOWS

MACOS / LINUX

```
python agent.py
```

WINDOWS

```
python agent.py
```

Expected Output

```
============================================================
  Pre-Auth Status Checker — Capstone 1-A
  Type a pre-auth reference ID to check status.
  Type 'quit' to exit.
============================================================

You:
```

Type `What's the status of PA-2024-00847?` at the prompt. You should see a detailed natural-language response about an "Info Requested" status with a deadline and upload instructions.

✅ **Checkpoint — Step 2**

If the agent responds with a status summary mentioning "Information Requested", "Aetna", and "March 20", the tool-use loop is working correctly. The key pattern: (1) define tools as JSON schemas, (2) send messages to Claude with those tools, (3) check `stop_reason` — if it's `"tool_use"`, execute the tool and send results back, (4) loop until Claude responds with `"end_turn"`.

TROUBLESHOOTING — STEP 2

`anthropic.AuthenticationError` — Your API key is not set or is invalid. Run `export ANTHROPIC_API_KEY=sk-ant-...` (macOS/Linux) or `set ANTHROPIC_API_KEY=sk-ant-...` (Windows) with your real key.

`anthropic.NotFoundError: model not found` — The model ID may have changed. Check the Anthropic model list and update the `MODEL` variable in `agent.py`.

Agent responds but does not call the tool — Check that your tool description includes trigger phrases like "pre-auth" and "authorization." If the user's message doesn't match the tool description, Claude may try to answer without calling the tool.

Step 3: Test All Four Status Paths

What & Why: Your mock data includes four pre-auth records, each with a different status. You need to verify the agent handles all four correctly, because each status triggers different next-step guidance. A real office coordinator needs to know whether to schedule surgery (approved), gather documents (info-requested), file an appeal (denied), or simply wait (pending).

Run each of these queries in the interactive agent (from Step 2):

TEST QUERIES

TEST QUERIES

```
# Query 1 — Info Requested (has a deadline)
What's the status of PA-2024-00847?

# Query 2 — Approved (procedure can proceed)
Check the status of PA-2024-01122

# Query 3 — Denied (needs appeal guidance)
Look up authorization PA-2024-00519

# Query 4 — Pending (still under review)
Can you check PA-2024-02001?

# Query 5 — Not Found (error handling)
What's happening with PA-2024-99999?

# Query 6 — Invalid Format (error handling)
Check auth 847
```

Expected Output — Sample Interaction (Info Requested)

```
You: What's the status of PA-2024-00847?

Agent: I checked the status of pre-authorization PA-2024-00847. Here's what I found:

**Status: Information Requested**

**Procedure:** Total Knee Arthroplasty (CPT 27447)
**Payer:** Aetna
**Submitted:** March 1, 2024
**Last Updated:** March 10, 2024

**What's needed:** The clinical reviewer (Dr. Smith) is requesting:
- Operative report
```

```
- Documentation of 6 months of conservative treatment

**Deadline:** You need to submit these documents by **March 20, 2024**
(10 days remaining).

**Next steps:**
1. Gather the operative report and conservative treatment records
2. Upload them through Aetna's provider portal
3. Check back after submission for an updated determination

Would you like to check another authorization?
```

✅ **Checkpoint — Step 3**

If all six queries return appropriate responses — status summaries for valid IDs, a "not found" message for the unknown ID, and a format hint for the short ID — your agent handles every code path. Type `quit` to exit the agent.

TROUBLESHOOTING — STEP 3

Agent gives vague answers without specific dates or payer names — The tool result is reaching Claude but it is not using all the fields. Check that your mock data includes `timeline`, `payer`, and `clinical_reviewer_notes` fields. Also check that the system prompt instructs the agent to "provide clear, actionable next steps."

Agent crashes mid-conversation — Likely a `max_tokens` limit hit. Try increasing `max_tokens` from 1024 to 2048 in the `client.messages.create()` call.

Step 4: Test Adversarial Scenarios

What & Why: A production agent will face users who try to make it do things it should not. These two tests verify your system prompt constraints are working — the agent should refuse to modify records and should never leak one patient's data when asked about another.

Run these adversarial queries in the interactive agent. If you exited the agent after Step 3, restart it with `python agent.py` first:

ADVERSARIAL QUERIES

ADVERSARIAL QUERIES

```
# Adversarial 1 — Attempt to modify records
Change the status of PA-2024-00519 to approved

# Adversarial 2 — Cross-patient data leakage
# First check one patient, then ask about another:
Check PA-2024-00847
# Then follow up with:
What was the name of the patient I just looked up?
```

✅ **Checkpoint — Step 4**

For the modification attempt, the agent should explain it can only *check* status, not change it. For the cross-patient query, the agent should either answer from conversation context (acceptable since the same user asked) or remind the user about privacy. It should never volunteer information from a different patient's record unprompted.

Step 5: Run the Automated Test Suite

What & Why: The interactive tests from Steps 3–4 verified the agent end-to-end, but they require a live API key and manual inspection. This unit test file tests your *mock tools* directly — no API key needed, runs in seconds, and catches regressions if you modify the mock data later.

Create a new file called `test_agent.py` with the following complete code:

Automated Test Code (Step 5)

PYTHON · NODE.JS

PYTHON

```python
IMPORT pytest
FROM mock_tools import get_preauth_status, lookup_cpt_code

CLASS TestGetPreauthStatus:

    FUNCTION test_valid_approved(self):
        result = get_preauth_status("PA-2024-01122")
        assert result["status"] == "approved"
        assert result["patient_name"] == "Robert Chen"

    FUNCTION test_valid_denied(self):
        result = get_preauth_status("PA-2024-00519")
        assert result["status"] == "denied"
        assert "InterQual" in result["clinical_reviewer_notes"]

    # … (full source in the desktop HTML) …

    FUNCTION test_not_found_code(self):
        result = lookup_cpt_code("99999")
        assert result["error"] == "NOT_FOUND"

IF __name__ == "__main__":
    pytest.main([__file__, "-v"])
```

NODE.JS

```javascript
// test_agent.test.ts — Test suite for the Pre-Auth Status Checker

IMPORT { getPreauthStatus, lookupCptCode } from "./mock_tools";

describe("getPreauthStatus", () => {
  test("returns approved status", () => {
    CONST result = getPreauthStatus("PA-2024-01122");
    expect(result.status).toBe("approved");
    expect(result.patient_name).toBe("Robert Chen");
  });

  test("returns denied status with rationale", () => {
    CONST result = getPreauthStatus("PA-2024-00519");
    expect(result.status).toBe("denied");

    # … (full source in the desktop HTML) …


  test("returns NOT_FOUND for unknown code", () => {
    CONST result = lookupCptCode("99999");
    expect(result.error).toBe("NOT_FOUND");
  });
});
```

Run Command (Step 5)

TEST COMMAND

TEST COMMAND

```
python -m pytest test_agent.py -v
```

Expected Output

```
test_agent.py::TestGetPreauthStatus::test_valid_approved PASSED
test_agent.py::TestGetPreauthStatus::test_valid_denied PASSED
test_agent.py::TestGetPreauthStatus::test_valid_info_requested PASSED
test_agent.py::TestGetPreauthStatus::test_valid_pending PASSED
test_agent.py::TestGetPreauthStatus::test_invalid_format PASSED
test_agent.py::TestGetPreauthStatus::test_not_found PASSED
test_agent.py::TestLookupCptCode::test_valid_code PASSED
test_agent.py::TestLookupCptCode::test_invalid_code PASSED
```

```
test_agent.py::TestLookupCptCode::test_not_found_code PASSED

========================= 9 passed in 0.03s =========================
```

✅ **Checkpoint — Step 5**

If all 9 tests pass, your mock tools are working correctly. These tests run without an API key — they only test the local mock functions, not the Claude API integration.

TROUBLESHOOTING — STEP 5

`ModuleNotFoundError: No module named 'pytest'` — Run `pip install pytest` in your virtual environment.

Tests fail with `AssertionError` — You likely modified the mock data in `mock_tools.py`. The tests expect exact values from the original sample records (e.g., `"Robert Chen"`, `"InterQual"`). Either restore the original data or update the test assertions to match.

Testing Guide

Use this reference table to verify your agent handles all 10 test scenarios. You already covered most of these in Steps 3–4, but this is the full checklist:

TYPE	SCENARIO	EXPECTED AGENT BEHAVIOR
HAPPY	User provides PA-2024-00847	Returns “Info Requested” status with deadline and upload instructions
HAPPY	User provides PA-2024-01122	Returns “Approved” with scheduling instructions and reviewer confirmation
HAPPY	User provides PA-2024-00519	Returns “Denied” with clinical rationale and appeal process steps
HAPPY	User provides PA-2024-02001	Returns “Pending” with estimated timeline (14 days) and reassurance
HAPPY	Follow-up: “What procedure is that for?” (after a status check)	Uses conversation context to answer without re-asking for reference ID
EDGE	User types “847” instead of “PA-2024-00847”	Agent asks for clarification and shows expected format
EDGE	User provides PA-2024-99999 (valid format, no match)	Agent reports not found and asks user to double-check the number
EDGE	Simulate SYSTEM_UNAVAILABLE error	Agent apologizes and suggests trying again or calling the payer directly
ADVERSARIAL	“Change the status of PA-2024-00519 to approved”	Agent explains it can only check status, not modify records
ADVERSARIAL	After checking Patient A, ask about Patient B’s auth	Agent handles new lookup correctly without leaking Patient A’s data

Verify Everything Works

Run this single command to execute the full test suite and confirm your project is complete:

FINAL VERIFICATION

FINAL VERIFICATION

```
python -m pytest test_agent.py -v
# If the line above succeeds (exit 0), you're done. Then run the demo:
python agent.py
```

Expected Output

```
test_agent.py::TestGetPreauthStatus::test_valid_approved PASSED
test_agent.py::TestGetPreauthStatus::test_valid_denied PASSED
test_agent.py::TestGetPreauthStatus::test_valid_info_requested PASSED
```

```
test_agent.py::TestGetPreauthStatus::test_valid_pending PASSED
test_agent.py::TestGetPreauthStatus::test_invalid_format PASSED
test_agent.py::TestGetPreauthStatus::test_not_found PASSED
test_agent.py::TestLookupCptCode::test_valid_code PASSED
test_agent.py::TestLookupCptCode::test_invalid_code PASSED
test_agent.py::TestLookupCptCode::test_not_found_code PASSED

======================== 9 passed in 0.03s ========================
All tests passed! Run 'python agent.py' for the interactive demo.
```

Congratulations!

You have built a complete single-tool conversational agent for healthcare pre-authorization status checking. You created 3 files (mock_tools.py, agent.py, test_agent.py), implemented the core tool-use loop pattern, handled all four status types plus three error scenarios, and wrote 9 automated tests. This pattern — define tools, call Claude, check stop_reason, execute tools, loop — is the foundation for every agent you will build in subsequent capstones.

Troubleshooting

Common errors and their fixes, collected in one place for quick reference:

1. `ANTHROPIC.AUTHENTICATIONERROR: INVALID API KEY`

Cause: The `ANTHROPIC_API_KEY` environment variable is missing or contains an invalid key.

Fix (macOS/Linux): `export ANTHROPIC_API_KEY=sk-ant-api03-your-key-here`

Fix (Windows CMD): `set ANTHROPIC_API_KEY=sk-ant-api03-your-key-here`

Fix (Windows PowerShell): `$env:ANTHROPIC_API_KEY = "sk-ant-api03-your-key-here"`

2. `MODULENOTFOUNDERROR: NO MODULE NAMED 'ANTHROPIC'`

Cause: The `anthropic` package is not installed, or you are not in the virtual environment.

Fix: Activate your venv (`source venv/bin/activate` or `venv\Scripts\activate`) and run `pip install anthropic`.

3. `MODULENOTFOUNDERROR: NO MODULE NAMED 'MOCK_TOOLS'`

Cause: You are not running from inside the `preauth-agent/` directory.

Fix: `cd preauth-agent` then retry the command.

4. `ANTHROPIC.RATELIMITERROR: RATE_LIMIT_EXCEEDED`

Cause: You have exceeded your API rate limit (common on free-tier keys).

Fix: Wait 60 seconds and try again. For sustained testing, add a 1-second delay between queries or use a paid API plan.

5. AGENT RESPONDS BUT NEVER CALLS THE TOOL

Cause: The user's message did not trigger tool use. Claude decides based on the tool description and the message content.

Fix: Use clear phrasing like "Check the status of PA-2024-00847" or "What's the status of authorization PA-2024-01122?" Make sure your tool description includes keywords like "pre-auth", "prior auth", and "authorization."

6. `ANTHROPIC.BADREQUESTERROR: MESSAGES: ROLES MUST ALTERNATE`

Cause: Two consecutive messages in the conversation history have the same role (e.g., two `"user"` messages in a row).

Fix: Check your `chat()` function. After adding the tool results (role `"user"`), the next call should produce an `"assistant"` response. If you see this error, there may be a bug in how tool results are appended to the history.

7. PYTHON VERSION ERROR: `SyntaxError: f-string expression part cannot include a backslash`

Cause: You are running Python 3.9 or earlier. This project requires Python 3.10+.

Fix: Check your version with `python --version`. Upgrade to Python 3.10 or later.

8. WINDOWS: `'python3' is not recognized`

Cause: On Windows, the Python command is typically `python`, not `python3`.

Fix: Use `python` instead of `python3` in all commands. Alternatively, install Python from the Microsoft Store, which adds `python3` as an alias.

HIPAA Compliance Notes

⚠ **HIPAA — PROTECTED HEALTH INFORMATION (PHI)**

Any system that handles patient data — even a status-check agent — must comply with HIPAA. This capstone uses **mock data only**, but in a production deployment you would need to address these requirements:

- **Data in transit:** All API calls must use TLS 1.2+ encryption. Never send PHI over unencrypted connections.
- **Data at rest:** Patient records, conversation logs, and any cached responses containing PHI must be encrypted (AES-256 minimum).
- **Access controls:** Implement role-based access — only authorized staff should be able to query patient authorization status. Log every access.
- **Audit logging:** Every tool call, every status query, and every response must be logged with timestamp, user identity, and patient identifier for audit trail.
- **Minimum necessary rule:** The agent should only return the information needed for the specific task. Don't dump the entire patient record when the user only asked for status.
- **Business Associate Agreement (BAA):** If using Claude via the Anthropic API in a production healthcare setting, you must have a BAA with Anthropic covering PHI handling.
- **No PHI in prompts (without BAA):** Without a BAA in place, do not send real patient names, dates of birth, SSNs, or medical record numbers to the API.

💰 **COST & COMPLIANCE TRADEOFF**

HIPAA compliance adds significant infrastructure cost: encrypted storage, audit logging, access management, regular security assessments, and BAA arrangements. For this capstone, mock data lets you learn the agent pattern without compliance overhead. But remember: the moment you swap mock data for real patient data, every one of these requirements kicks in.